

Experiment 1: Understanding Precision, Recall and F1 (Manual Calculation)

Objective

Understand how evaluation metrics are calculated from a confusion matrix.

Python Program

```
import numpy as np

# Example Confusion Matrix values
TP = 40
FP = 10
FN = 5
TN = 45

precision = TP / (TP + FP)
recall = TP / (TP + FN)
f1 = 2 * precision * recall / (precision + recall)
accuracy = (TP + TN) / (TP + TN + FP + FN)

print("Evaluation Metrics")
print("-" * 30)

print(f"True Positive : {TP}")
print(f"False Positive: {FP}")
print(f"False Negative: {FN}")
print(f"True Negative : {TN}")

print("\nCalculated Metrics")

print(f"Accuracy : {accuracy:.3f}")
print(f"Precision: {precision:.3f}")
print(f"Recall : {recall:.3f}")
print(f"F1 Score : {f1:.3f}")
```

Learning Outcome

Students learn:

- Confusion Matrix
 - Accuracy
 - Precision
 - Recall
 - F1 Score
 - Mathematical formulas behind evaluation
-

Experiment 2: Evaluate Predictions using Scikit-learn

Objective

Use built-in functions to evaluate classification results.

Install

```
pip install scikit-learn
```

Python Program

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# True labels
y_true = [1,0,1,1,0,1,0,0,1,1]

# Model predictions
y_pred = [1,0,1,0,0,1,1,0,1,1]

print("Accuracy :", accuracy_score(y_true,y_pred))

print("Precision:", precision_score(y_true,y_pred))

print("Recall  :", recall_score(y_true,y_pred))

print("F1 Score :", f1_score(y_true,y_pred))

print("\nConfusion Matrix")

print(confusion_matrix(y_true,y_pred))

print("\nClassification Report")

print(classification_report(y_true,y_pred))
```

Sample Output

```
Accuracy : 0.80
Precision: 0.83
Recall: 0.83
F1 Score: 0.83
```

Confusion Matrix

```
[[3 1]
 [1 5]]
```

Learning Outcome

Students understand

1. Automated evaluation
2. Classification Report
3. Support
4. Macro Average
5. Weighted Average

Experiment 3: Sentiment Classification using Hugging Face + Evaluation

Objective

Evaluate predictions from a Transformer model.

Install

```
pip install transformers torch scikit-learn
```

Python Program

```
from transformers import pipeline
from sklearn.metrics import classification_report
```

```
classifier = pipeline(
    "sentiment-analysis",
    model="distilbert-base-uncased-finetuned-sst-2-english"
)
```

```
texts = [
    "I love this movie.",
    "This product is terrible.",
    "Excellent service.",
    "Worst experience ever.",
    "Very happy with the purchase.",
    "I hate waiting."
]
```

```
true_labels = [
    "POSITIVE",
    "NEGATIVE",
    "POSITIVE",
    "NEGATIVE",
    "POSITIVE",
    "NEGATIVE"
]
```

```
predictions = classifier(texts)

predicted_labels = [item["label"] for item in predictions]

print("Predictions")

for text,pred in zip(texts,predicted_labels):
    print(f'{text:35} --> {pred}')

print("\nClassification Report\n")

print(classification_report(true_labels,predicted_labels))
```

Learning Outcome

Students learn

- Evaluate a pretrained Transformer
- Compare predicted vs actual labels
- Precision
- Recall
- F1 Score

Experiment 4: Confusion Matrix Visualization

Objective

Visualize model performance.

Install

```
pip install matplotlib scikit-learn
```

Python Program

```
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.metrics import confusion_matrix

y_true = [
    "Positive", "Negative", "Positive", "Negative",
    "Positive", "Negative", "Positive", "Positive"
]

y_pred = [
    "Positive", "Negative", "Negative", "Negative",
    "Positive", "Positive", "Positive", "Positive"
]

cm = confusion_matrix(y_true,y_pred)

disp = ConfusionMatrixDisplay(
```

```
confusion_matrix=cm,  
display_labels=["Negative", "Positive"]  
)  
  
disp.plot(cmap="Blues")  
  
plt.title("Confusion Matrix")  
plt.show()
```

Learning Outcome

Students understand

1. True Positives
2. False Positives
3. False Negatives
4. Visual evaluation of model predictions

Suggested Hands-on Exercises

1. Change the prediction labels and observe how **Precision**, **Recall**, and **F1-Score** change.
2. Create an **imbalanced dataset** (e.g., 90% positive, 10% negative) and compare **Accuracy** with **F1-Score** to understand why accuracy alone can be misleading.
3. Evaluate multiple pretrained sentiment models (e.g., DistilBERT vs. BERT) on the same dataset and compare their classification reports.
4. Modify the input text (e.g., adding negation such as "not good") and analyze how the model's predictions and evaluation metrics change.
5. Plot confusion matrices for different models and discuss common error patterns.

Expected Learning Outcomes

After completing these four experiments, students will be able to:

1. Explain the significance of **Accuracy**, **Precision**, **Recall**, and **F1-Score**.
2. Compute evaluation metrics manually and using **Scikit-learn**.
3. Evaluate predictions from a **Transformer-based** sentiment classifier.
4. Interpret **classification reports** and **confusion matrices**.
5. Compare different NLP models using standard evaluation metrics commonly used in research and industry.